

MASTER TECHNICAL REFERENCE · OWNER EDITION

ScamGraph 마스터 레퍼런스

실시간 사기·피싱 위협 인텔리전스 웹 플랫폼 — 프로젝트의 **모든 구성 요소가 무엇이고 어떻게 동작하는지**를 소유자가 완전히 이해하고, 심사위원의 어떤 질문에도 답할 수 있도록 만든 기술 도시에(dossier).

- 순수 SW · AI 없음
- 설명 가능
- 데모 세이프
- 폴리글랏 모노레포
- SDG 16 · 10

대상: 프로젝트 소유자(발표자) · 심사 대비
범위: 웹 · 게이트웨이 · 엔진 · 브라우저 확장 · 모바일 앱 · 인프라 · 법률/개인정보
구성: 10개 Docker 서비스 · 폴리글랏(TypeScript · Java · Python · Dart/Kotlin)
문서 버전: 데모 빌드 v0.1 기준 · 생성 2026-07

1. 목차

1. 개요 · 비전 · 핵심 원칙

한 줄 정의와 문제의식
4대 불변 원칙
무엇이 다른가(차별점)

2. 아키텍처

폴리글랏 구성과 스택
데이터 흐름 다이어그램
10개 서비스 · 포트 맵
계약 기반 병렬 개발

3. 컴포넌트별 "어떻게 동작하는가"

규칙 엔진 `quick_assess` — 무네트워크 즉시 판정
심층 수집 `crawl_and_enrich` + `score_enrichment`
정확도 평가 하네스
그래프 귀속(Neo4j) · 그래프 분석
위협 피드 파이프라인(TIP)
게이트웨이 — 통합 두뇌
웹 — 관제 대시보드
브라우저 확장 — 개인정보 보존형 방화벽
모바일 앱 — 전화·문자 보호 계층
커뮤니티 플라이휠 · 설명 가능성 · 데모 세이프

4. API 레퍼런스 (전 엔드포인트)

5. 킬샷 end-to-end 메커니즘

6. 실행 · 데모 방법

기동 명령
90초 데모 스크립트
트러블슈팅

7. 테스트 · CI · 부하

8. 법률 · 개인정보 · 악용 방지

9. 설계 결정과 이유 · 예상 Q&A

2. 개요 · 비전 · 핵심 원칙

2.1 한 줄 정의와 문제의식

ScamGraph는 의심스러운 **URL · 전화번호 · 계좌번호**를 넣으면 규칙 엔진과 공개 데이터로 **위험도와 그 근거**를 즉시 설명하고, 수집된 사기 인프라(도메인·IP·전화·계좌)를 **Neo4j 관계망**으로 이어 "사기 조직"을 시각화하는 실시간 위협 인텔리전스 웹 플랫폼이다.

메이커톤 주제 "**디지털 시민으로서 안전한 디지털 세상**"(SDG 16 평화·정의·제도, SDG 10 불평등 완화 — 정보 취약계층 보호)에 대한 출품작이다. 보이스피싱·스미싱·피싱 사이트는 **여러 채널(웹·문자·전화)에서 동시에, 조직적으로** 이뤄진다. 단일 URL 검사기로는 "이 링크가 위험하다"까지만 말하지만, ScamGraph는 **같은 IP·인증서·전화·계좌를 공유하는 캠페인 전체**를 드러내고, 그 판정을 **브라우저 확장과 모바일 앱으로 배포**해 다음 피해자를 선제적으로 막는다.

2.2 4대 불변 원칙

2.2.1 순수 SW · AI 금지

모든 위험 판정은 **명시적 규칙**으로만 이뤄진다. 블랙박스 신경망이 없다. "왜 위험한가"를 언제나 재현·설명할 수 있는 것이 세일즈 포인트다.

2.2.2 설명 가능성(reasons)

모든 판정에 {rule, weight, detail, confidence?} 근거 배열이 붙는다. 점수만 단독 노출하지 않는다. UI·PDF·알림 어디서든 근거가 함께 간다.

2.2.3 데모 세이프

외부 네트워크·DB·브로커가 **모두 죽어도** API와 화면은 시드/폴백으로 즉시 응답한다.

`quick_assess` 는 **절대 네트워크를 호출하지 않는다**.

2.2.4 프론트는 게이트웨이만 호출

웹은 엔진을 직접 부르지 않는다. 모든 접점(웹·확장·모바일)은 게이트웨이의 통합 엔드포인트 하나만 알면 된다 = "공용 두뇌, 얇은 클라이언트".

2.3 무엇이 다른가(차별점)

ScamGraph는 "피싱 URL 검사기"가 아니라 네 가지를 동시에 보여준다:

- ① **실시간 다중채널 예방** — 웹 스캔 + 브라우저 확장(DNR 로컬 차단) + 모바일(전화·문자 스크리닝)이 하나의 판정을 공유.
- ② **범죄 인프라 상관분석** — 공유 IP/인증서/등록자로 흩어진 도메인을 하나의 캠페인으로 묶는 그래프 귀속.
- ③ **시민 신고 공유 방어망** — 커뮤니티 신고가 쌓이면 전 사용자가 보호되는 플라이휠(단, poisoning 방어 필수).
- ④ **블랙박스 AI 없는 설명가능 보안 제품** — 모든 판정이 규칙·증거로 재현되고, 오탐은 이의제기·정정 피드로 회수된다.

3. 아키텍처

3.1 폴리글랏 구성과 스택

의도적으로 **언어별 강점에 맞춰 서비스를 분리**한 폴리글랏 마이크로서비스 모노레포다. "학생이 못 만들 규모"로 보이게 하는 것도 전략의 일부다(심사 = 비전문가 상호투표).

레이어	스택	역할
web	Next.js 15 · React 19 · TypeScript	다크 "관제실" 대시보드 — 스캔 콘솔, Sigma.js 관계망, deck.gl 지도, WebSocket 실시간 피드
gateway	Java 21 · Spring Boot(가상 스레드)	API 게이트웨이 · 통합 판정 두뇌 · Swagger · WebSocket 허브 · 블록리스트 배포 · 레이트리밋
engine	Python 3.12 · FastAPI	규칙 엔진(<code>quick_assess</code>) · 크롤링/enrich · 그래프 적재 · 검색 색인
worker	Python · Celery(+beat)	비동기 크롤링 · 위협 피드 주기 수집 · Neo4j 적재
neo4j	Neo4j 5 Community(+APOC)	사기 인프라 관계망 그래프 DB
postgres	PostgreSQL 16	스캔·신고·블록리스트·구독·알림·이의제기·API키 메타
redis	Redis 7	Celery 브로커/결과 백엔드 · 캐시
meilisearch	Meilisearch v1.10	위협 엔티티 자체 검색엔진
prometheus	Prometheus v3.1	게이트웨이·엔진 메트릭 수집(관측성 스택)
grafana	Grafana 11.4	메트릭 대시보드(익명 접속 허용 — 데모 세이프)

3.2 데이터 흐름 다이어그램



읽는 법

실선은 요청 방향, 게이트웨이가 유일한 접점이다. 웹·확장·모바일은 게이트웨이만 호출하고, 게이트웨이가 엔진 (`ENGINE_URL=http://engine:8000`)으로 위임한다. 그래프/피드 조회는 게이트웨이가 **Neo4j**를 직접 읽되 (`GraphSource`) 비어 있으면 **시드 그래프로 폴백**한다.

3.3 10개 서비스 · 포트 맵

`docker compose up --build` 한 번으로 아래 10개 컨테이너가 함께 뜬다. 관측성 스택(prometheus-grafana)은 기존 8개 서비스 동작에 영향을 주지 않는 부가 서비스다.

서비스	호스트 포트	확인 URL / 용도	비고
web	3001 → 3000	관제 대시보드	호스트 3000 점유 대비 3001 매핑
gateway	8080	API · Swagger /docs	모든 클라이언트의 단일 접점
engine	8000	엔진 API /docs	--reload 볼륨 마운트
worker	—	Celery(+내장 beat)	포트 없음
neo4j	7474 / 7687	브라우저 / Bolt	neo4j / scamgraph123 · APOC
postgres	내부 5432	메타/신고 DB	호스트 미노출(내부 전용)
redis	내부 6379	브로커/캐시	호스트 미노출
meilisearch	7700	검색엔진	MEILI_MASTER_KEY
prometheus	9090	메트릭 수집	보존 6h
grafana	3003 → 3000	메트릭 대시보드	익명 접속 허용(데모)

포트 주의

README/스펙 문서는 대시보드를 :3000 으로 안내하지만, 실제 `docker-compose.yml` 은 web을 호스트 3001 , grafana를 3003 에 매핑한다(호스트 3000 충돌 회피). 로컬 `npm run dev` 로 프론트만 띄우면 :3000 이다. 심사장에서 열 주소는 **localhost:3001** .

3.4 계약 기반 병렬 개발

세 파트(엔진 / 게이트웨이+웹 / 인프라)의 유일한 접점은 **API 계약**이다. 응답 JSON 스키마(특히 `/graph` 의 `nodes/edges` , `grade ∈ safe|caution|warning|danger`)만 지키면 서로 기다리지 않고 병렬로 개발할 수 있다. 스키마를 바꾸면 상호 협의가 원칙이다. 실제로 이 프로젝트는 다수의 트랙(엔진 정확도·피드·그래프분석·PDF·알림·신고방어·웹·확장·모바일·인프라·법률)으로 병렬 실행됐다.

4. 컴포넌트별 "어떻게 동작하는가"

이 장이 이 문서의 핵심이다. 각 부분이 **구체적으로 어떤 메커니즘으로** 동작하는지 — 신호 하나하나의 가중치, 폴백 경로, 데이터 모양까지 — 설명한다.

4.1 규칙 엔진 `quick_assess` — 무네트워크 즉시 판정

파일: `apps/engine/app/crawler.py`. 입력(URL/전화/계좌)을 받아 **네트워크를 전혀 건드리지 않고** 위험 점수(0–100)와 근거 배열을 즉시 반환한다. 데모 세이프의 심장이다. `tldextract` 는 `suffix_list_urls=()` · `cache_dir=None` 으로 생성해 **패키지 번들 스냅샷만** 쓰고 네트워크/디스크 접근을 원천 차단한다.

4.1.1 처리 순서

- 입력 분류** `classify_target()`: 영문자/./ / 포함 → `url`; 숫자 9–11자리 → `phone`; 10–16자리 → `account`.
- 화이트리스트 즉시 안전**: 호스트가 `ALLOWLIST` (약 40개 정상 도메인, 하위 도메인 포함)면 즉시 **safe** 0점 반환. 반드시 원본 호스트로 검사한다 — 키릴 위장 `naver.com` 은 스켈레톤이 `naver.com` 이라도 통과하면 안 되기 때문.
- 혼동문자 접기**: `decode_idna()` 로 퓨니코드를 유니코드로 되돌리고 `confusable_skeleton()` 으로 ASCII 스켈레톤을 만들어 토큰·브랜드 분석을 견고하게 한다.
- eTLD+1 분해**: `tldextract` 로 `co.kr·go.kr` 같은 멀티파트 TLD까지 정확히 분해(`_registrable`). 실패 시 `labels[-2]` 근사로 폴백 → 절대 예외를 던지지 않는다.
- 규칙 누적 채점**: 아래 신호들을 순차 적용해 점수를 더하고 각 `reason` 을 남긴다. 최종 `min(score,100)`.

4.1.2 URL 신호 카탈로그 (가중치·신뢰도)

rule	탐지 대상	가중치	conf
<code>verified_domain</code>	화이트리스트 정상 도메인 → 즉시 안전 종료	0	—
<code>homograph</code>	퓨니코드(<code>xn--</code>) 도메인 사용 자체	+35	0.7
<code>homoglyph</code>	키릴/그리스/전각 혼동문자·혼합 스크립트 위장. 스켈레톤이 화이트리스트와 일치(표적 위장)면 +50	+40 / +50	0.9 / 0.97
<code>brand_impersonation</code>	등록 도메인에 브랜드명 포함(정확일치·임베드)하나 공식 아님	+35	0.8
<code>typosquatting</code>	브랜드와 Levenshtein 편집거리 ≤2 (토큰·브랜드 4자↑, 길이차 ≤2, 거리2는 6자↑ 브랜드만)	+38	0.85
<code>brand_subdomain</code>	브랜드가 서브도메인에만 있고 실제 등록 도메인은 다름(목적지 위장)	+30	0.8
<code>suspicious_tld</code>	위험 TLD(<code>zip·mov·top·xyz·click·tk·ml·cf·ga·loan...</code> 20종)	+22	—
<code>phishing_keywords</code>	유도 키워드(<code>secure·login·otp·delivery·refund·police·subsidy...</code> 약 60종). 다중 히트 가중(상한 30)	+16~30	—
<code>digit_heavy</code>	도메인 이름에 숫자 4개 이상(자동 생성 흔적)	+10	—
<code>hyphen_heavy</code>	하이픈 3개 이상(키워드 조합 위장)	+10	—
<code>ip_host</code>	도메인 대신 점10진 IP 주소 사용	+30	0.85
<code>obfuscated_ip</code>	IP를 정수/16진으로 인코딩(목적지 은폐)	+30	0.9

rule	탐지 대상	가중치	conf
<code>url_shortener</code>	알려진 단축 서비스(실제 목적지 은폐 — 양날이라 보수적)	+20	0.5
<code>at_symbol</code>	URL 내 @ (실제 목적지 은폐 기법)	+25	0.7
<code>double_encoding</code>	이중 퍼센트 인코딩(<code>%25XX</code> — 필터 우회)	+18	0.7
<code>nonstandard_port</code>	비표준 포트(80/443 아님)	+12	0.45
<code>deep_subdomain</code>	과도한 서브도메인 깊이(라벨 5개↑)	+15	—
<code>long_url</code>	비정상적으로 긴 URL(75자↑)	+10	—
<code>no_tls</code>	스킴이 있으나 HTTPS 미사용	+8	—

전화 신호: `voip_prefix` (070/050 인터넷전화, +20), `intl_prefix` (선행 + 또는 00 국제발신, +15). **계좌:** `account_lookup` (가중치 0 — 커뮤니티 신고 DB 대조 필요 표시).

4.1.3 점수 → 등급 매핑

`_grade(score)`: **danger** ≥ 70 **warning** ≥ 35 **caution** ≥ 15 **safe** < 15 . 표현 가이드상 **safe** 는 "안전 보장"이 아니라 **"아직 악성 미확인"**으로 서술한다.

```
>>> quick_assess("shinhan-otp.xyz")
{ "kind": "url", "risk_score": 73, "grade": "danger",
  "reasons": [
    {"rule": "brand_impersonation", "weight": 35, "confidence": 0.8, "detail": "'shinhan' 브랜드명이 도메"},
    {"rule": "suspicious_tld", "weight": 22, "detail": "위험 TLD '.xyz'"},
    {"rule": "phishing_keywords", "weight": 16, "detail": "피싱 유도 키워드: otp"} ] }
```

왜 이게 중요한가

이 함수는 **네트워크·DB·브로커가 전부 죽어도** 밀리초 단위로 근거와 함께 결과를 낸다. 심사장 와이파이가 끊겨도 킬샷은 항상 동작한다. "AI 없이 규칙만으로도 이만큼 정확하고 설명 가능하다"가 핵심 메시지다.

4.2 심층 수집 `crawl_and_enrich` + `score_enrichment`

`quick_assess` 가 즉답을 낸 뒤, Celery 워커가 **best-effort로 실제 네트워크 수집**을 수행한다. 모든 호출은 `try/except` + 타임아웃으로 감싸 실패해도 부분 결과를 반환한다(예외 없음).

수집	방법(라이브러리·타임아웃)	추출 신호
리다이렉트/콘텐츠	<code>httpx follow_redirects, timeout 6s</code>	<code>final_url · redirects[]</code> · 상태코드·HTML
폼 분석	<code>BeautifulSoup</code> (<code>_analyze_content</code>)	비밀번호 input·OTP/보안카드 텍스트·외부 도메인 form action
WHOIS	<code>python-whois</code>	도메인 생성일(나이)·등록자(귀속 pivot)
DNS	<code>dnspython</code> A레코드, lifetime 5s	호스팅 IP 목록
TLS	<code>ssl/socket</code> :443 (<code>_inspect_tls</code>)	발급자·유효기간·SHA-256 지문 16자·검증 실패 여부

`score_enrichment()` 가 수집 신호를 추가 채점한다: `newly_registered` (30일 미만 +25), `recent_domain` (90일 미만 +12), `redirect_chain` (2회↑ +12), `credential_form` (비번폼 +20),

otp_form (+22), external_form (외부전송 +15), tls_invalid (+12), new_cert (14일 미만 발급 +10). 이렇게 규칙 신호(quick)와 인프라 신호(deep)를 분리해 "quick→deep 재판정"을 UI로 보여줄 수 있다.

SSRF·데모 세이프

심층 수집은 워커에서만, 사용자가 명시 실행한 스캔에 한해 동작한다. 크롤러는 private IP·metadata endpoint 차단 등 SSRF 방어가 로드맵상 필수 요건이다. quick_assess 경로에는 어떤 네트워크 호출도 없다.

4.3 정확도 평가 하네스

4.4 그래프 귀속(Neo4j) · 그래프 분석

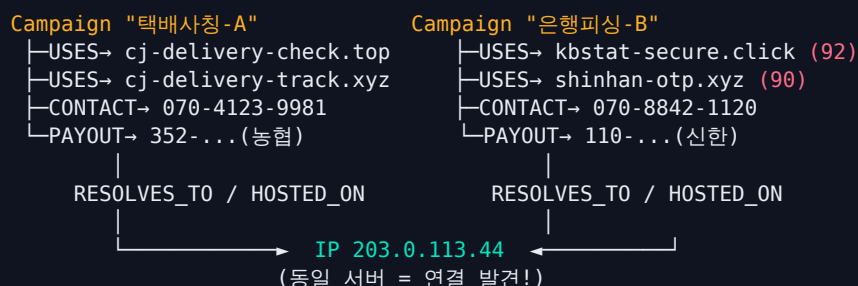
ScamGraph의 하이라이트는 "공유 인프라로 흩어진 도메인을 하나의 조직으로 잇는" 귀속이다. 엔진 (app/graph.py)이 스캔 결과를 Neo4j에 upsert 하고, 시드(infra/neo4j/seed.cypher)가 데모용 관계망을 미리 적재한다.

4.4.1 노드·관계 스키마

노드 라벨	속성	관계 타입
Campaign	name, type	USES (캠페인 → 타겟) · RESOLVES_TO (타겟 → 호스트) · HOSTED_ON (호스트 → IP) · CONTACT (캠페인 → 전화) · PAYOUT (캠페인 → 계좌) · REPORTS (신고 → 타겟)
Target	value, kind, grade, risk_score, last_seen	
Host	name	
IP	addr	
Phone	number, carrier	
Account	number, bank	
Report	source, note, ts	

4.4.2 귀속의 핵심 통찰 — "같은 서버 = 동일 조직"

시드 그래프는 이 통찰을 시각적으로 증명한다. 서로 다른 두 캠페인이 같은 IP 203.0.113.44 를 공유한다:



게이트웨이의 CheckController.findOrganization() 는 입력값을 호스트로 정규화해 그래프 노드와 매칭한 뒤, BFS(너비우선탐색)로 가장 가까운 Campaign 노드를 찾아 조직명을 돌려준다. 캠페인에 연루되면 규칙 점수와 무관하게 등급을 danger 로 올린다("인프라 연루 = 규칙보다 강한 신호"). 이렇게 /api/campaign 계열로 "사건 파일"을 복원한다.

4.4.3 그래프 분석 알고리즘

4.5 위협 피드 파이프라인(TIP)

4.6 게이트웨이 — 통합 두뇌

게이트웨이(`apps/gateway` , Java 21 · Spring Boot, 가상 스레드)는 모든 접점의 단일 접점이자 "공용 두뇌"다. 핵심은 **통합 판정** `GET /api/check` 이다.

4.6.1 통합 판정 `GET /api/check?value=`

브라우저 확장·PWA·모바일 앱이 **이 엔드포인트 하나만** 알면 되도록, 규칙 판정(엔진) + 조직 귀속(그래프) + 커뮤니티 신고 + 외부 피드 대조 + 실행 권고를 한 번에 합성한다. 각 단계는 실패해도 다음으로 넘어가는(`try/catch`) 데모 세이프 구조다.

단계	동작	등급 영향
0) 입력 검증	길이 > 2048자 → <code>grade:"unknown"</code> 즉시 반환	방어
1) 규칙 판정	엔진 <code>POST /scan</code> 호출 → <code>kind·grade·risk_score·reasons</code>	기본 등급
2) 조직 귀속	<code>GraphSource</code> BFS로 최근접 Campaign 탐색	연루 시 → danger
2b) 커뮤니티 신고	<code>SELECT COUNT(*) FROM reports WHERE target=?</code>	≥3 → danger, ≥1 → warning
2c) 외부 피드 대조	<code>blocklist</code> 조회(value 또는 host) → <code>external_feed_hit</code> (weight 40, <code>source·first_seen</code>)	등재 시 → danger
3) 실행 권고	<code>kind·grade·org</code> 별 문구 + 커뮤니티/피드 접두	—

```
GET /api/check?value=shinhan-otp.xyz →
{ "value":"shinhan-otp.xyz", "kind":"url", "grade":"danger", "risk_score":73,
  "reasons":[ ...엔진 근거..., {"rule":"external_feed_hit","weight":40,"source":"openphish",...}
  "organization":"은행피싱-B", "community_reports":2, "feed_sources":["openphish"],
  "recommendation":"🚨 위협 피드 등재(openphish). 👥 커뮤니티 2건 신고됨. 🚨 위협 - 링크를 누르지 말고
```

설명 가능성 유지

외부 피드 대조도 "어느 피드에 언제 등재됐는지"를 `reason` 으로 남긴다. 게이트웨이는 판정을 합성하되 근거를 버리지 않는다.

4.6.2 신고 poisoning 방어 — `ReportDefense`

커뮤니티 플라이휠은 강력하지만 동시에 공격 표면이다. 한 행위자가 무고한 도메인을 대량 허위신고해 `danger` 로 승격시키면 업무방해·괴롭힘이 된다. 오염 지점은 `/api/check` 가 `COUNT(*) FROM reports` 로 신고 수를 읽어 등급을 올리는 곳이므로, **접수 단계에서 게이팅**해 `reports` 에는 "dedup을 통과한 독립 신고자의 첫 기여"만 쌓이게 한다 → 원시 행수 = 독립 신고자 수 → "한 명이 아무리 반복해도 1".

계층	메커니즘	상수
신고자 익명 해시	헤더(X-Forwarded-First-IP / X-Client-Id / XFF / remoteAddr)에서 식별자를 뽑아 pepper + SHA-256 . 원시 IP는 저장 안 함	<code>REPORT_HASH_SALT</code>

계층	메커니즘	상수
Dedup	동일 신고자·동일 대상은 전 기간 1회만 집계	—
Rate limit(burst)	시간창 내 독립 신고 상한 초과 시 throttle	창 3600s / 상한 20
독립 신고자 카운트	등급을 좌우하는 수 = <code>COUNT(DISTINCT reporter_hash)</code>	다수 신고 임계 3
Allowlist 가드	정상 도메인(엔진과 동일 집합) 신고는 <code>status='review'</code> 검토 큐로만, 자동 승격 금지	—

승격 정책: ① 1건=저장만 → ② 독립 다수(≥ 3)="다수 신고" → ③④⑤ 위험후보·확인·전체 blacklist는 **강한 기술 신호 결합 + 운영자 dual-approval**이 있어야 하며 **신고 접수 경로에서 자동 승격하지 않는다** (`blockEscalationEligible` 혹). DB가 죽어도 정상 도메인만은 자동 승격을 막는 폴백을 낸다.

4.6.3 WebSocket 실시간 피드 허브 — `/ws/feed`

`FeedHandler` (`TextWebSocketHandler`)가 접속한 모든 세션에 스캔·신고 이벤트를 브로드캐스트한다. 실이벤트가 없어도 **4초마다 시드 더미 이벤트를 순환** (`@Scheduled(fixedRate=4000)`)해 피드가 항상 "살아있게" 한다(데모 세이프). 다른 빈이 `broadcast()` 로 실이벤트를 주입한다. 죽은 세션 하나가 루프 전체를 깨지 않도록 격리한다.

```
// /ws/feed 이벤트 예 (4초 하트비트 순환)
{ "type":"report", "target":"cj-delivery-check.top", "kind":"url",
  "grade":"danger", "risk_score":92, "note":"택배 미수령 사칭 문자", "ts":1720... }
```

4.6.4 그래프 소스 · 블록리스트 · 그 외 컨트롤러

`GraphSource` 는 Neo4j 실데이터를 우선 읽되(`Neo4jGraphReader`, 500노드), 비었거나 오류면 **SeedGraph** 로 폴백한다. 여러 엔드포인트가 매 요청 호출하므로 **8초 TTL 캐시**로 Neo4j 부하를 줄인다.

4.7 웹 — 관제 대시보드

4.8 브라우저 확장 — 개인정보 보존형 방화벽

4.9 모바일 앱 — 전화·문자 보호 계층

4.10 커뮤니티 플라이휠 · 설명 가능성 · 데모 세이프

4.10.1 커뮤니티 플라이휠

시민 신고 → 독립 다수 검증 → 강한 기술신호와 결합 → 확인된 위협 → 서명 블록리스트 → 확장·모바일 배포 → 다음 사용자 보호. 신고가 쌓일수록 방어망이 커진다.

4.10.2 설명 가능성

모든 판정에 `reasons`. 사실(feed·기술신호)/추정(그래프 상관)/신고(시민)를 **색+아이콘+텍스트**로 분리 표기. 신고만으로 danger 금지.

4.10.3 데모 세이프

3중 폴백: ① `quick_assess` 무네트워크 ② `GraphSource` → `SeedGraph` ③ WS 4초 하트비트. 엔진·Neo4j·PG가 모두 죽어도 화면이 산다.

5. API 레퍼런스 (전 엔드포인트)

6. 킬샷 end-to-end 메커니즘

데모의 심장: **스캔 입력** → **위험도+근거** → **관계망 그래프가 뻗어 나가는** 장면. 이 한 흐름이 탐지·설명가능성·그래프 귀속·실시간을 전부 전달한다. 아래는 각 단계에서 **어떤 데이터가 오가는지**다.

```

사용자 입력  shinhan-otp.xyz
|
| ① ScanConsole (웹, React) - 게이트웨이만 호출
|
|
| GET /api/check?value=shinhan-otp.xyz (게이트웨이 · CheckController)
|
| ② 엔진 POST /scan → quick_assess (무네트워크, ~100ms)
|    grade=danger, risk_score=73, reasons=[brand_impersonation, suspicious_tld, phishing_keywo
|
| ③ GraphSource BFS → 최근접 Campaign "은행피싱-B" (organization)
|
| ④ reports COUNT · blacklist 대조(external_feed_hit)
|
|
| 응답 JSON { grade, risk_score, reasons[], organization, community_reports, feed_sources, recommendati
|
| ⑤ 웹: 위험 게이지 + 근거 카드 렌더
|
|
| GET /api/graph/expand?value=shinhan-otp.xyz (해당 노드 N-hop 이웃)
|
| ⑥ nodes/edges 반환 → Sigma.js 그래프에 새 노드 애니메이션 추가
|
|
| 그래프가 공유 IP 203.0.113.44 를 통해 기존 캠페인으로 뻗어 나감 ← 킬샷
|
| ⑦ 카메라가 새 노드로 포커스 · deck.gl 지도 좌표 표시 · WS 피드에 이벤트
|
|
| (선택) Case File PDF 생성 · 커뮤니티 신고 수 증가
  
```

6.1 단계별 데이터

단계	주체	오가는 데이터
① 입력·전송	웹 ScanConsole → 게이트웨이	GET /api/check?value=<입력>. 웹은 엔진을 직접 호출하지 않는다.
② 규칙 판정	게이트웨이 → 엔진 /scan	quick_assess 가 무네트워크로 {kind, risk_score:73, grade:"danger", reasons[3]} 반환
③ 조직 귀속	게이트웨이 GraphSource	호스트로 정규화 후 BFS → 최근접 Campaign "은행피싱-B" → 연루 시 등급 danger 고정
④ 신고·피드	게이트웨이 → Postgres	reports COUNT(≥3 danger/≥1 warning) + blacklist 대조 (external_feed_hit weight 40, source.first_seen)
⑤ 판정 렌더	웹	위험 게이지 + 근거 카드(사실/추정/신고 등급 배지) + 권고 문구
⑥ 그래프 확장	웹 → 게이트웨이 /api/graph/expand	입력 노드 기준 이웃 {nodes, edges} → Sigma.js(graphology forceatlas2)로 새 노드·엣지 추가
⑦ 스펙터클	웹	카메라 포커스 애니메이션 · deck.gl 지도 · /ws/feed 이벤트 스트림

왜 관객이 놀라는가

②는 100ms 내 근거와 함께 뜨고(설명가능), ⑥은 **흩어진 도메인이 같은 IP를 공유하는 캠페인으로 뻗어 나가는 순간**을 시각화한다(귀속). 두 장면이 "단순 검사기"와 "위협 인텔리전스 플랫폼"의 차이를 5초 만에 증명한다. 그리고 네트워크가 끊겨도 `SeedGraph · quick_assess` 폴백으로 **항상** 재현된다.

7. 실행 · 데모 방법

7.1 기동 명령

명령	동작
<code>make up</code>	<code>docker compose up --build</code> — 10개 서비스 전체 빌드·기동
<code>make seed</code>	Neo4j 시드 주입(<code>cypher-shell ... seed.cypher</code>) — Neo4j 뜬 뒤 실행
<code>make logs</code> / <code>make ps</code>	로그 팔로우 / 상태 확인
<code>make down</code> / <code>make clean</code>	중지 / 볼륨까지 삭제
<code>make loadtest</code>	k6 부하 테스트(Docker, host 네트워크, <code>BASE_URL=:8080</code>)

```
# 프론트만 빠르게 (게이트웨이는 docker로 띄운 채)
cd apps/web && npm install && npm run dev                # :3000

# 규칙 엔진만 즉시 테스트 (의존 서비스 불필요)
python -c "from app.crawler import quick_assess; print(quick_assess('shinhan-otp.xyz'))"
```

확인 URL: 대시보드 `:3001` (로컬 dev는 `:3000`) · 게이트웨이 Swagger `:8080/docs` · 엔진 `:8000/docs` · Neo4j `:7474` (neo4j/scamgraph123) · Grafana `:3003` .

환경변수 (선택)

실제 위협 피드 키(`ABUSECH_AUTH_KEY` , `DATA_GO_KR_KEY`)는 없어도 된다 — 없으면 **시드 풀백**으로 동작(데모 세이프). 프론트는 `NEXT_PUBLIC_GATEWAY_URL` (기본 `http://localhost:8080`).

7.2 90초 데모 스크립트 — "한 흐름으로 전부 전달"

탐지·그래프·실시간·멀티플랫폼·사회적 가치를 **하나의 스토리**로 보여주는 순서다.

- [0:00] 관제 화면 오픈** — 상단 티커가 흐르고 서비스 LED가 순차 점등(부팅 시퀀스), 상태바 "SYSTEM ONLINE". "이건 실시간으로 도는 관제 플랫폼입니다."
- [0:10] 스캔 입력** — 스캔 콘솔에 `shinhan-otp.xyz` 입력. ~100ms 내 **danger 73** + 근거 카드 3개 (`brand_impersonation·suspicious_tld·phishing_keywords`)가 뜬다. "AI가 아니라 규칙이 왜 위험한지 설명합니다."
- [0:25] 킬샷 — 그래프가 뻗는다** — 판정과 동시에 관계망 그래프에 입력 노드가 추가되고, **공유 IP 203.0.113.44** 를 통해 기존 캠페인('은행피싱-B')으로 **엣지가 뻗어 나간다**. 카메라가 새 노드로 포커스. "흠어진 도메인이 같은 조직임을 드러냅니다."
- [0:45] 실시간·다중채널** — 우측 라이브 피드에 신고/스캔 이벤트가 4초마다 흐른다. deck.gl 지도에 위협 좌표 표시. "브라우저 확장과 모바일이 같은 판정을 공유합니다."
- [1:00] 사회적 가치 · 사건 파일** — 커뮤니티 신고 수 증가, 통합 판정 권고 문구("링크를 누르지 마세요"). 필요 시 경찰 연계용 Case File PDF 생성. "한 시민의 신고가 전 사용자를 보호합니다."
- [1:20] 마무리** — "탐지 + 그래프 귀속 + 실시간 공유 방어 + 설명가능성, 블랙박스 AI 없이."

데모 세이프 보증

위 모든 장면은 외부 네트워크가 끊겨도 동작한다: 스캔은 `quick_assess` (무네트워크), 그래프는 `SeedGraph` 폴백, 피드는 4초 하트비트, 지도/통계는 시드 데이터. **라이브에서 실제로 동작**하되 절대 깨지지 않는다.

7.3 트러블슈팅

증상	원인 · 조치
대시보드가 안 열림	포트 확인 — compose는 web을 <code>:3001</code> 에 매핑(3000 아님). Grafana는 <code>:3003</code> .
그래프가 비어 보임	<code>make seed</code> 미실행. 단, 시드 없어도 <code>SeedGraph</code> 폴백으로 기본 그래프는 나온다.
스캔 결과가 안 나옴	엔진이 죽어도 게이트웨이가 그래프/피드 기반 폴백 판정을 낸다. 완전 실패면 <code>grade:"unknown"</code> .
피드가 멈춘 듯	WS 재연결 대기. 하트비트가 4초 주기이므로 잠시 후 재개.
게이트웨이 빌드 실패	로컬 JDK 25면 컨테이너 빌드 권장(타겟 JDK 21). <code>docker compose up --build gateway</code> .
429 Too Many Requests	IP당 60초 600건 레이트리밋. 정상 데모 트래픽에선 발생하지 않음.

8. 테스트 · CI · 부하

9. 법률 · 개인정보 · 악용 방지

ScamGraph는 전화·계좌·도메인을 위험도와 함께 다룬다. 그런데 **이 식별자의 소유자가 곧 범죄자는 아니다** (발신번호 조작·대포통장 명의도용·도메인 침해). 따라서 표현·개인정보·악용방지가 제품의 필수 방어선이다.

문서: `docs/{legal-privacy,expression-guidelines,abuse-defense,dpia-checklist}.md`.

면책

관련 문서는 엔지니어링 내부 설계 기준이며 법률 자문이 아니다. **출시 전 반드시 한국 변호사·개인정보 전문가의 실제 법률검토**(형법 명예훼손·업무방해, PIPA, 정보통신망법, 신용정보법, 통신비밀보호법, 전기통신사업법, 위치정보법)를 받아야 한다. 검토 전 실서비스 공개 배포 금지. 해커톤 데모는 **시드/폴백 데이터**로 운영해 실제 개인 식별자 노출을 피한다.

9.1 법적 리스크 매트릭스(요약)

리스크	근거 법령	완화책
개인 명예훼손(발신번호 명의자를 범인으로 노출)	형법 §307 · 정보통신망법 §70	단정 금지 표현 · 마스킹 · 명의자 미수집
사업자 업무방해(침해된 정상 사이트를 루트째 차단)	형법 §314	서브도메인 탈취 시 루트 전체 악성판정 금지 · 이의제기 · 긴급 정정 delta
부정확 정보 확산(오탐이 검색·공유로 확산)	PIPA(정확성) · 민사 손해배	<code>noindex</code> · 자동 마스킹 · tombstone · 오탐 신고 채널
통신 내용 침해(SMS 원문·통화 서버 전송)	통신비밀보호법	원문 기기 밖 미전송 · URL/indicator만
국외이전 위반(외부 feed·SaaS로 개인정보 이전)	PIPA §28-8	hash lookup 우선 · 이전 고지 · 최소화

9.2 개인정보 최소화 — "수집하지 않은 데이터는 유출될 수 없다"

데이터	처리
SMS/문자 원문	서버 미전송 — 기기 내 로컬 규칙엔진에서 URL/도메인만 추출·전송
통화 내용/녹음	미수집 — 번호·행동 신호만
전화번호	서버 조회는 HMAC/keyed-hash lookup(원문 최소)
계좌번호 / 명의자	마스킹 + keyed hash / 명의자 미수집
정확 위치(GPS)	미수집 — 지도는 GeolIP 국가 집계만, 오차 표기
방문 기록(브라우저)	미전송 — 확장은 로컬 blocklist로 판정
검사 기록 · 신고자 신원	opt-in 저장 / 신고자는 어디에도 미공개

9.3 표현 가이드 — 단정 금지

UI·API·PDF·알림 어디서든 **동일한 표현 규칙**을 적용한다. 3대 원칙: ① 단정 금지("사기범/사기 계좌" 금지) ② 사실/추정/신고 분리(색+아이콘+텍스트) ③ 근거·출처·관측시각·규칙버전 4요소 동반.

금지(단정)	승인(관측·정황)
"이 번호는 사기범입니다"	"사기 신고에 사용된 것으로 관측된 발신번호 (발신번호는 조작 가능)"
"사기 계좌", "이 계좌 주인은 사기범"	"신고 및 연관 정황이 있는 계좌 (명의자 ≠ 수익자일 수 있음)"
"이 사이트 운영자는 사기범"	"피싱에 사용된 것으로 관측된 도메인"
"동일 사기 조직"	"동일 캠페인 84% 추정 (공유 근거 4개)" · 캠페인은 코드명 CAMP-YYYY-NNN

증거 등급(evidence tier): 사실(confirmed — feed·기술신호, 실선 배지) / 추정(inferred — 그래프 상관, 점선) / 신고(reported — 시민, 물음표). 공개 화면은 마스킹(전화 뒤 4자리·계좌 뒤 2자리·URL **defang** `hxxp / [.]`)·`noindex`·reverse lookup 금지.

9.4 악용 방지 — 신고 승격 정책

기본 태도: **1건의 신고는 신호일 뿐, 판정이 아니다.** 판정은 다층 검증 승격을 통해서만 상향된다(구현: §3.6 ReportDefense).

단계	트리거	결과 · 승인
① 저장만	신고 1건	판정 변경 없음 · 자동
② 다수 신고 표시	독립 신고자 다수(≥3)	caution 신호 가산 · 자동
③ 위험 후보	신고 + 강한 기술신호 결합	warning 후보 · 자동+사후검토
④ 확인된 위협	신뢰 feed 확인 또는 검증출처(경찰/은행)	danger · 자동(검증출처)
⑤ 전체 blocklist	운영자 검토 + 복수 강증거	서명 blocklist 배포 · 운영자 dual-approval 필수

이의제기·정정: 누구나(비로그인) 접근 가능한 appeal form → 소유권 확인 → (피해 크면)임시 suppress → 판정 변경/기각 → **긴급 정정 delta로 확장·모바일에서 회수** · tombstone · 색인/캐시 purge. **서비스 역이용 방어:** 익명 결과는 요약만(내부 임계값 비공개), 세부 그래프는 인증 조사자만(public/partner/internal 증거 레벨 분리), bulk API 승인제, 크롤러 SSRF/격리.

9.5 투명성

정기 투명성 보고서(`docs/transparency-report-template.md`)로 판정 정책·승격 기준(요지)·feed 출처·삭제/정정 건수·이의제기 인용률을 공개해 신뢰를 확보한다. 개인정보 영향평가(DPIA) 체크리스트도 별도 문서로 관리한다.

10. 설계 결정과 이유 · 예상 Q&A

10.1 핵심 설계 결정과 근거

결정	왜
폴리글랏 10서비스 (TS·Java·Python·Dart)	언어별 강점 배치(프론트 TS·게이트웨이 동시성 Java 가상스레드·규칙/크롤 Python) + 심사 (비전문가 상호투표)에서 "규모 있어 보임"이 전략적 이점
AI 금지, 순수 규칙	설명 가능성이 세일즈 포인트 · GPU/학습데이터 불필요 · 데모 세이프(무네트워크 즉답) · 법적 방어(왜 위험한지 재현 가능)
quick_assess 무네트워크	심사장 와이파이가 끊겨도 킬샷 동작 · 밀리초 응답 · <code>tlldextract</code> 번들 스냅샷으로 네트워크 원천 차단
프론트는 게이트웨이만	"공용 두뇌, 얇은 클라이언트" — 확장·모바일·웹이 <code>/api/check</code> 하나만 알면 됨 · 정책 일관성 · 엔진 교체 자유
그래프 귀속(Neo4j)	"이 링크 위험"을 넘어 "같은 조직"을 드러냄 = 진짜 차별점 · 공유 IP/인증서/전화/계좌가 pivot
블록리스트 배포(서버 URL 조회 X)	방문기록을 서버에 안 보냄(개인정보) · 오프라인 차단 · 서버 장애 무관 · SHA-256 서명 스냅샷+delta
신고 poisoning 방어	플라이휠 무결성 · 대량 허위신고 업무방해 방지(형법 §314) · 독립 신고자만 집계
표현 가이드·마스킹	발신번호 조작·대포통장·도메인 침해 → 소유자#범죄자 · 명예훼손(형법 §307) 리스크 차단
모든 곳에 시드/폴백	데모 세이프 — 외부 의존이 다 죽어도 화면이 산다(3중 폴백)
등급 임계값 70/35/15	화이트리스트(약 40개 정상 도메인)와 결합해 오탐을 억제하도록 튜닝 — 정상 도메인은 즉시 <code>safe</code>

10.2 심사위원 예상 질문 & 답변

10.2.1 Q. "AI가 아니라면서 어떻게 정확한가요?"

A. 24개 이상의 명시적 신호(브랜드 사칭, Levenshtein 타이포스쿼팅, UTS#39 혼동문자, 유니코드, 인코딩 IP, 위험 TLD, 피싱 키워드, URL 구조)를 가중 합산합니다. 정상 도메인 화이트리스트로 오탐을 억제하고, 라벨링된 표본으로 평가 하네스가 정확도를 측정합니다(→ §7). AI가 아니라서 **왜 그 점수인지 근거를 항상 재현**할 수 있는 게 오히려 강점입니다.

10.2.2 Q. "오탐이 나면 어떻게 하나요?"

A. 세 단계입니다. ① 사전: 정상 도메인 화이트리스트로 즉시 `safe`. ② 사후: 누구나 접근 가능한 이의제기 폼. ③ 회수: 확정 시 **긴급 정정 delta**로 확장·모바일에서 즉시 rule 제거 + tombstone + 색인 purge. 침해된 정상 사이트는 루트 전체를 악성으로 단정하지 않습니다.

10.2.3 Q. "무고한 사람을 사기범으로 몰아 명예훼손 아닌가요?"

A. 발신번호는 조작 가능하고 대포통장은 명의도용이며 도메인은 침해될 수 있어 **소유자 ≠ 범죄자**입니다. 그래서 "사기범" 단정을 금지하고 "관측된/신고된/정황이 있는" 프레임만 씁니다. 식별자는 마스킹.defang하고 계좌 명의자는 수집하지 않으며 상세 페이지는 `noindex` 입니다.

10.2.4 Q. "개인정보를 침해하지 않나요?"

A. "수집하지 않은 데이터는 유출될 수 없다"가 원칙입니다. SMS 원문·통화 내용은 기기 밖으로 나가지 않고 (URL/지표만), 방문기록은 서버에 안 보내며(확장은 로컬 blocklist로 판정), 전화/계좌는 hash lookup·마스킹만 합니다.

10.2.5 Q. "네트워크가 끊기면 데모가 망하지 않나요?"

A. 3중 폴백입니다. 스캔은 무네트워크 `quick_assess`, 그래프는 `SeedGraph` 폴백, 실시간 피드는 4초 하트 비트, 통계/지도는 시드. 외부 의존이 전부 죽어도 화면이 살아 있습니다.

10.2.6 Q. "경쟁사가 대량 신고로 정상 사이트를 차단시킬 수 있나요?"

A. `ReportDefense` 가 막습니다. 신고자 익명 해시로 dedup(한 명이 반복해도 1), 시간창 burst 제한, 독립 신고자 수(`DISTINCT`)로만 집계, 정상 도메인 신고는 검토 큐로만. 신고 1건이 단독으로 `danger` /blocklist가 되는 일은 없고 ⑤단계는 운영자 dual-approval 필수입니다.

10.2.7 Q. "학생 프로젝트치고 규모가 큰데 실제로 도나요?"

A. 네. `docker compose up` 하나로 10개 서비스가 뜨고, 자동화 테스트와 k6 부하 테스트(→ §7)가 있으며, 브라우저 확장과 모바일 앱까지 같은 판정을 공유합니다. 모든 킬샷은 시드/폴백으로 **라이브에서 항상 동작**합니다.

10.2.8 Q. "블록리스트는 어떻게 배포하나요?"

A. 게이트웨이가 정규화된 지표를 SHA-256 해시·버전과 함께 스냅샷/델타로 제공하고, 확장은 이를 `declarativeNetRequest` 로컬 규칙으로 변환해 **브라우저가 방문기록을 서버에 보내지 않고 즉시 차단**합니다. 모바일도 오프라인 blocklist로 같은 방식으로 동작합니다.